11 Komunikace v síti - tvorba síťových aplikací, Berkley socket a jeho rozhraní

TCP

Většinou pro komunikaci v síti používám TCP. Abychom si tyto pakety mohli posílat bezestrátově, používáme TCP. Všechny packety a data se ověřují přes handshake.

Začíná tím, že před spojením naváže spojení. Poté zjistí, jestli někdo poslouchá. Pošlou se mu Ackleknowledge, ten odpoví zpátky. Poté se pošle request a nakonec se zašle

TCP Umí vytvořit posloupnost dat. Data se odesílají v nějakém pořadí. Dokáže ztracené packety poslat znovu.

Zkontrolujeme, jestli nějaké chybí a poté je zašleme znovu. TCP nemá Broadcasting, teda rozesílání na více adres.

TCP je mnohem pomalejší, ale poskytuje všechna data (někdy žádná)

TCP se nejvíce využívá třeba v emailech, hledání na stránkách – nejširší využití se používá na webu, základ HTTP protokolu je TCP protokol.

TCP vždy posílá na jednu stranu data, strana pošle zpátky ackleknowledge.

Mezi výhody TCP patří to, že pracuje bez řešení operačního systému, dá se optimalizovat rychlost.

Jeho nevýhody jsou to, že je pomalejší a když nepřijdou nějaká data, tak nepřijdou vůbec žádná.

U TCP je možnost, že někdy přijdou data lehce přeházená, ale vždy se nakonec složí.

UDP

UDP protokol posílá data rozházeně, některé mohou třeba chybět. U UDP se mohou nějaké ztratit a není zaručené jejich doručení.

UDP broadcasting má – to znamená, že může rozeslat packety na více zařízení.

UDP je velmi rychlé, ale je riziko ztracených packetů.

UDP je skvělé pro vysílání live a nebo při hraní her, kde záleží na rychlosti. Posílá velice rychle a nečeká na potvrzení. Hlasové volání, skype, hovory a podobné.

Neexistuje systém na potvrzení doručení dat.

V Pythonu

V Pythonu používáme knihovnu Socket. Vytvoříme instanci, tudíž Berkeleyeho socket a necháme ho poslouchat. Jakmile se uživatel připojí, pošlemu mu nějakou zprávu a poté ho necháme zadávat nějaké zprávy a tiskneme si je do konzole (Ano, můžeme je ukládat jinak normálně do souboru nebo je zpracovat). Jakmile uživatel zadá „konec“, tak se connection ukončí.

```python
import socket

addr = ("127.0.0.1", 65525)  # nastavení informací o naší adrese, na které pojede server
srv_socket = socket.socket()  # vytvoření socketu, na který se mohou lidé připojit
srv_socket.bind(addr)  # přidáme socketu naší adresu
srv_socket.listen()  # necháme ho poslouchat na tomto portu

while True:  # nekonečný cyklus, jenž se snaží navázat spojení s tím kdo se připojí
    try:
        conn, client_ip = srv_socket.accept()  # jakmile se někdo připojí, vytvoří connection a uloží i jeho adresu
        conn.send(bytes("Ahoj", "utf-8"))  # zašle mu zprávu ahoj
        conn.settimeout(30)  # nastavíme timeout po třiceti vteřinách bez odpovědi
        while True:
            try:
                data = conn.recv(1024).decode("utf-8").strip().lower()  # načteme prvních 1024 bytů od uživatele 1kb
                print(data)  # data vytisknu
                if data == "konec":
                    conn.close()
            except:
                pass
    except:
        pass
```